

Dynamical Estimation Suite

User's Manual

MIT 2.c01/2.c51 Physical Systems Modeling and Design Using Machine Learning

Sensitivity and Adjoint Methods for Parameter Estimation in Dynamical Systems

Directed by Prof. George Barbastathis; executed by Claude

The suite implements gradient-descent estimation of the parameter vector $\mathbf{w}$ and initial condition $\mathbf{x}_0$ for a dynamical system

$$\dot{\mathbf{x}}(t) = \mathbf{h}(\mathbf{x}, t; \mathbf{w}), \quad \mathbf{x}(0) = \mathbf{x}_0,$$

from a time-series dataset $\{t_n, \mathbf{x}_n\}_{n=1}^N$ by minimising the ℓ_2 loss $\mathcal{E} = \frac{1}{2N} \sum_n \|\epsilon_n\|^2$. Two equivalent algorithms are provided: the *sensitivity method* (reliable, cost $\propto KP$ per iteration) and the *adjoint method* (efficient, cost $\propto K$ per iteration). The second method, for better numerical stability, has been implemented with continuous gradient accumulation and checkpointing.

MATLAB Suite

Requirements and file structure

MATLAB R2018b or later; no additional toolboxes required. Place all .m files on the MATLAB path.

File	Purpose
dyn_forward.m	Forward ODE solve; stores all internal RK45 steps
dyn_residuals.m	Residuals ϵ and loss function $\mathcal{E}$
dyn_sensitivity.m	Augmented (x, s, s_{ic}) ODE
dyn_adjoint.m	Backward ODE with gradient accumulator and checkpointing
dyn_update_sensitivity.m	Parameter update via sensitivity
dyn_update_adjoint.m	Parameter update via adjoint
dyn_estimate_sensitivity.m	Full sensitivity estimation loop
dyn_estimate_adjoint.m	Full adjoint estimation loop
dyn_run.m	Grand pipeline (calls all of the above)
dyn_plot_forward.m	Plot trajectory + data
dyn_plot_sensitivity.m	Plot $\mathbf{s}(t)$ or $\mathbf{s}_{ic}(t)$
dyn_plot_adjoint.m	Plot $\mathbf{a}(t)$ with jump gaps
dyn_plot_residual_force.m	Stem plot of $\mathbf{v}(t)$
dyn_plot_loss.m	Loss history vs. iteration
lv_h.m, lv_J.m, lv_U.m, lv_test.m	Lotka–Volterra example

Defining your dynamical system

Provide three function handles with the calling convention $\mathbf{f}(\mathbf{x}, \mathbf{t}, \mathbf{w})$, where $\mathbf{x}$ is $(K \times 1)$, $\mathbf{t}$ is a scalar, and $\mathbf{w}$ is $(P \times 1)$.

```
h_fun    = @(x, t, w) ...; % (K x 1) plant operator h(x, t; w)
J_fun    = @(x, t, w) ...; % (K x K) Jacobian dh/dx
U_fun    = @(x, t, w) ...; % (K x P) plant sens. dh/dw
U_ic_fun = [];             % (K x K) pass [] if, as is usual,
                           % h does not depend on x0
```

For the Lotka–Volterra example (lv_h.m etc.) with $\mathbf{w} = [w_{11}, w_{13}, w_{21}, w_{23}]^\dagger$:

```
function dx = lv_h(x, t, w)
dx = [w(1)*x(1) - w(2)*x(1)*x(2);
      -w(3)*x(2) + w(4)*x(1)*x(2)];
end
```

Quick start: running dyn_run

```

t_data = linspace(0.5, 20, 120); % sampling times

% Stopping: stop if windowed (W=15) mean loss decreases by < 1e-6
plot_opts.idx_state = [1, 2]; % state components to plot
plot_opts.idx_sens = [1 1; 1 2]; % [k,p] pairs for S (1-based)
plot_opts.idx_sens_ic = []; % skip S_ic plot
plot_opts.idx_adj = [1, 2];
plot_opts.idx_resi = [1, 2];
plot_opts.log_scale = true;
plot_opts.fig_offset = 1;

results = dyn_run(h_fun, J_fun, U_fun, U_ic_fun, ...
w_true, x0_true, w_init, x0_init, [], ... % S0=[]
t_data, alpha, beta, ...
M, 1e-6, 15, false, ... % M, E_frac_stop, M_check_stop, stop_on_increase
'my_results.mat', [], ... % save path, ode_opts ([] = defaults)
true, plot_opts); % verbose, plot_opts

```

Key function signatures

Signature	Returns
<code>dyn_forward(h,w,x0,t_data)</code>	<code>x_tdata(K×N)</code> , <code>t_sol(1×S)</code> , <code>x_sol(K×S)</code>
<code>dyn_residuals(x_tdata, x_data)</code>	<code>resi(K×N)</code> , <code>loss</code>
<code>dyn_sensitivity(h,J,U,Uic,w,x0,S0,t)</code>	<code>S_tdata(K×P×N)</code> , <code>S_ic_tdata</code> , <code>x_tdata</code> , <code>t_sol</code> , <code>x_sol</code> , <code>S_sol(K×P×S)</code> , <code>S_ic_sol</code>
<code>dyn_adjoint(J,U,w,t_sol,x_sol,t,resi,...h,x0,x_tdata)</code>	<code>a_tdata(K×N)</code> , <code>a_0(K×1)</code> , <code>grad_w(P×1)</code> , <code>t_adj</code> , <code>a_adj</code>
<code>dyn_estimate_sensitivity(...)</code>	<code>w_hat(P×1)</code> , <code>x0_hat(K×1)</code> , <code>loss_hist</code>
<code>dyn_estimate_adjoint(...)</code>	<code>w_hat</code> , <code>x0_hat</code> , <code>loss_hist</code>

Stopping criteria

Both estimation functions accept four stopping arguments (after M):

<code>E_frac_stop</code>	Fractional-decrease threshold; 0 disables early stopping.
<code>M_check_stop</code>	Window size W ; stop if $(\bar{\mathcal{E}}[m-W:m-1] - \bar{\mathcal{E}}[m-W+1:m]) / \bar{\mathcal{E}}[m-W:m-1] < \text{E_frac_stop}$. Set 1 for the single-step criterion.
<code>stop_on_increase</code>	If <code>false</code> (default), windowed loss increases trigger a warning but do not stop the loop — useful for studying the effect of noisy data or large learning rates when the gradient descent misbehaves.

Output files

`dyn_run` writes three outputs alongside the `.mat` file:

- `my_results.mat` — `results` struct with all trajectories, sensitivities, adjoints, loss histories, and estimates.
- `my_results.tex` — booktabs table comparing ground truth, sensitivity estimate, and adjoint estimate.
- `figures/my_results*.png` — one 300 dpi PNG per figure (10 files in a full verbose run, fewer if `idx_sens_ic` is empty).

Python Suite

Requirements and package structure

Python 3.8+, with `numpy`, `scipy`, and `matplotlib`. Install dependencies via `pip install numpy scipy matplotlib`.

File	Purpose
<code>dyn.suite/__init__.py</code>	Package entry point; re-exports all public functions
<code>dyn.suite/forward.py</code>	<code>dyn_forward</code> , <code>dyn_residuals</code>
<code>dyn.suite/sensitivity.py</code>	<code>dyn_sensitivity</code> , <code>dyn_update_sensitivity</code>
<code>dyn.suite/adjoint.py</code>	<code>dyn_adjoint</code> , <code>dyn_update_adjoint</code>
<code>dyn.suite/estimation.py</code>	<code>dyn_estimate_sensitivity</code> , <code>dyn_estimate_adjoint</code>
<code>dyn.suite/run.py</code>	<code>dyn_run</code> grand pipeline
<code>dyn.suite/plotting.py</code>	All five <code>dyn_plot.*</code> functions
<code>lv.system.py</code>	Lotka–Volterra <code>lv_h</code> , <code>lv_J</code> , <code>lv_U</code>
<code>lv.test.py</code>	Test driver (mirrors <code>lv.test.m</code>)

Defining your dynamical system

Function calling conventions are identical to MATLAB; all arrays are `numpy` arrays.

```
import numpy as np

def my_h(x, t, w):          # returns (K,) array
    return np.array([...])

def my_J(x, t, w):          # returns (K, K) array
    return np.array([[...]])

def my_U(x, t, w):          # returns (K, P) array
    return np.array([[...]])

U_ic_fun = None             # pass None if h does not depend on x0
```

Quick start: running `dyn_run`

```
from dyn_suite import dyn_run
import numpy as np

t_data = np.linspace(0.5, 20.0, 120)

plot_opts = dict(
    idx_state = [0, 1],          # 0-based state indices
    idx_sens   = [(0,0), (0,1)], # (k, p) pairs, 0-based
    idx_sens_ic = None,          # None to skip
    idx_adj    = [0, 1],
    idx_resi   = [0, 1],
    log_scale  = True,
    fig_offset = 1,
)

results = dyn_run(
    my_h, my_J, my_U, None,      # U_ic_fun=None
    w_true, x0_true, w_init, x0_init, None, # S0=None
    t_data, alpha, beta,
    M=600, E_frac_stop=1e-6, M_check_stop=15, stop_on_increase=False,
    save_path='my_results',      # writes .pkl, .tex, figures/
    rtol=1e-8, atol=1e-10,
    verbose=True, plot_opts=plot_opts)
```

Key function signatures

Signature	Returns
<code>dyn_forward(h,w,x0,t_data)</code>	<code>x_tdata(K,N)</code> , <code>t_sol(S,)</code> , <code>x_sol(K,S)</code>
<code>dyn_residuals(x_tdata, x_data)</code>	<code>resi(K,N)</code> , <code>loss</code>
<code>dyn_sensitivity(h,J,U,Uic,w,x0,S0,t)</code>	<code>S_tdata(K,P,N)</code> , <code>S_ic_tdata</code> , <code>x_tdata</code> , <code>t_sol</code> , <code>x_sol</code> , <code>S_sol(K,P,S)</code> , <code>S_ic_sol</code>
<code>dyn_adjoint(J,U,w,t_sol,x_sol,t,resi,...h,x0,x_tdata)</code>	<code>a_tdata(K,N)</code> , <code>a_0(K,)</code> , <code>grad_w(P,)</code> , <code>t_adj</code> , <code>a_adj</code>
<code>dyn_estimate_sensitivity(...)</code>	<code>w_hat(P,)</code> , <code>x0_hat(K,)</code> , <code>loss_hist</code>
<code>dyn_estimate_adjoint(...)</code>	<code>w_hat</code> , <code>x0_hat</code> , <code>loss_hist</code>

Differences from the MATLAB suite

Aspect	MATLAB	Python
Indices	1-based	0-based
Optional functions	Pass []	Pass None
Result storage	<code>struct (.mat)</code>	<code>dict (.pkl)</code>
Results access	<code>results.w_hat_s</code>	<code>results['w_hat_s']</code>
Sensitivity idx	[1 1; 1 2] matrix	[(0,0), (0,1)] list of tuples
Array shapes	($K \times N$) matrices	(K, N) NumPy arrays
ODE options	<code>odeset('RelTol',...)</code>	<code>rtol=...</code> , <code>atol=...</code> <code>kwargs</code>

Output files

`dyn_run` writes outputs when `save_path` is not None:

- `my_results.pkl` — Python dict with all results, loadable with `pickle.load(open('my_results.pkl','rb'))`.
- `my_results.tex` — same `booktabs` table as the MATLAB version.
- `figures/my_results_*.png` — 300 dpi PNGs saved via `matplotlib`; one per quantity (same naming convention as MATLAB).

Suite developed in the context of the MIT class 2.c01/2.c51, sponsored by the Schwartzman College of Computing.

Both platforms implement identical mathematics; numerical results may differ slightly due to floating-point and ODE-solver differences, but should agree to within solver tolerance when initialized from the same starting point.